



Vendor Specific Information

Cisco IOS is the operating system of Cisco network devices such as routers and switches. It provides features and services required to manage and operate network devices. This operating system comes in different versions and releases that vary in features, support, and performance. It offers several features required for the operation of modern networks, such as, but not limited to:

- Support for IPv6
- Quality of Service (QoS)
- Security features such as encryption and authentication
- Virtualization features such as Virtual Private LAN Service (VPLS)
- Virtual Routing and Forwarding (VRF)

Cisco IOS can be managed in several ways, depending on the network device and hardware used. The most commonly used method is the command line interface (**CLI**), which can also be managed in the graphical user interface (**GUI**). In addition, it supports various network protocols and services required for network operations. These include:

Protocol Type	Description
Routing protocols	Such as OSPF and BGP are used to route data packets on a network.
Switching protocols	Such as VLAN Trunking Protocol (VTP) and Spanning Tree Protocol (STP) is used to configure and manage switches on a network.
Network services	Such as Dynamic Host Configuration Protocol (DHCP) are used to automatically provide clients on the network with IP addresses and other network configurations.
Security features	Such as Access Control Lists (ACLs), which are used to control access to network resources and prevent security threats.

In Cisco IOS, different types of passwords are used for various purposes, for example:

Password Type	Description
User	The user password is used for logging in to Cisco IOS. It is used to restrict access to the network device and its features.
Enable Password	The enable password is used to enter "enable" mode. The "enable" mode is the mode where you have access to advanced functions and settings.

Password Type	Description
Secret	The secret is a password to secure access to certain functions and services. It is often used to restrict access to remote management tools and services.
Enable Secret	The enable secret is an extra-secure password used to secure access to "enable" mode, and they are stored encrypted to provide additional protection.

We highly recommend going through the provided external resources to understand the encryption mechanics of Cisco IOS and how those are used.

The Cisco IOS devices can be configured for SSH or Telnet. So it can be accessed remotely. We can determine from the response we receive that it is indeed a Cisco IOS, as it responds with the **User Access Verification** message.

Cisco IOS

```
athane@htb[/htb]$ telnet 10.129.10.2
```

shellsession

```
Trying 10.129.10.2...
Connected to 10.129.10.2.
Escape character is '^]'.
```

User Access Verification

Password:

VLANs

Imagine this scenario: A startup called XQ hired a network administrator to create a network for their single-office company, and due to budget limitations, they can only afford one switch and router. The sysadmin of XQ stated that in addition to hosting the web and database servers in the network, staff from different departments will be using it. As a seasoned network security specialist, the network administrator immediately thought about the security attacks that an insider can perform, especially ones abusing broadcast traffic, such as **broadcast storms**. Therefore, to tackle this problem, the network administrator decided to logically segment the network with **Virtual Local Area Networks** (**VLANs**), conceptually breaking down one switch into smaller mini-switches.

A **VLAN** is a logical grouping of network endpoints connected to defined ports on a switch, allowing the segmentation of networks by creating logical broadcast domains that can span multiple physical LAN segments. With **VLANs**, network administrators can segment networks based on factors such as team, function, department, or application, without worrying about the physical location of endpoints and users. A broadcast packet sent over one **VLAN** does not reach any other endpoint that is a member of another **VLAN**. Because each **VLAN** is regarded as a broadcast domain, it needs to have its own **subnet**; for example, the network administrator contracted by XQ can segment the network by departments:

Department	VLAN ID	Subnet
Servers	VLAN 10	192.168.1.0/24
C-Level	VLAN 20	192.168.2.0/24
Finance	VLAN 30	192.168.3.0/24
HR	VLAN 40	192.168.4.0/24
Marketing	VLAN 50	192.168.5.0/24
Support	VLAN 60	192.168.6.0/24

A myriad of benefits is attained when using **VLANs** , including:

- **Better Organization** : Network administrators can group endpoints based on any common attribute they share.
- **Increased Security** : Network segmentation disallows unauthorized members from sniffing network packets in other **VLANs** .
- **Simplified Administration** : Network administrators do not have to worry about the physical locations of an endpoint.
- **Increased Performance** : With reduced broadcast traffic for all endpoints, more bandwidth is made available for use by the network.

Cisco switches provide the **VLAN** IDs/numbers 1-4094 (**0** and **4095** are reserved IDs and cannot be used); IDs 1-1005 (**VLAN 1** is known as the **default VLAN** and cannot/should not be altered nor deleted) are known as **normal-range VLANs**, with IDs 1002-1005 being reserved for **Token Ring** and **Fiber Distributed Data Interface (FDDI) VLANs**, while IDs 1006-4094 are known as **extended-range VLANs**. By default, any customization applied for **normal-range VLANs** is saved in the **VLAN** database (the **vlan.dat** file), in contrast to **extended-range VLANs**, which do not have their customizations saved. **VLANs** 2-1001 stored in **vlan.dat** can have parameters including name, type, state, and maximum transmission unit (**MTU**).

VLAN Memberships

Network administrators can assign the ports of a switch to **VLANs** either statically or dynamically. Static **VLAN** assignment, which is the simplest and most common method, involves assigning each port to a **VLAN** manually using the switch's **network operating system**; this must be done for all switches separately (it is essential to keep in mind that endpoints connecting to these ports are unaware of the existence of **VLANs**). In contrast, dynamic **VLAN** assignment automatically determines an endpoint's **VLAN** membership based on **MAC** addresses or protocols. The system administrator can register the **MAC** addresses in a centralized **VLAN** management service/database, such as the **VLAN Membership Policy Server (VMPS)** service, and then the switch queries the database of **VMPS** to determine the **VLAN** of the endpoint with that specific **MAC** address. Regardless of their flexibility and mobility, dynamic **VLANs** increase administrative overhead.

Security-wise, static **VLANs** are the more secure option because a port will forever be tied to a specific **VLAN** ID, unless changed manually afterward. For dynamic **VLANs**, an attacker could

potentially utilize tools such as **macchanger** to spoof the MAC address of legitimate endpoints and attain membership of their **VLANs**, therefore sniffing all network traffic sent through them.

Access and Trunk Ports

Any port on a **VLAN**-enabled switch must be either an **access port** or a **trunk port**. **Access ports** belong to and can carry the traffic of only one **VLAN** (or in some cases two, with the second being for **voice traffic**); any traffic arriving on an **access port** is assumed to belong to the **VLAN** the port was assigned. On the other hand, **trunk ports** can carry multiple **VLANs** at the same time; **trunk links** connect two **trunk ports** on two switches (or a switch and router) to allow information from multiple **VLANs** to be carried out across switches.

VLAN Identification

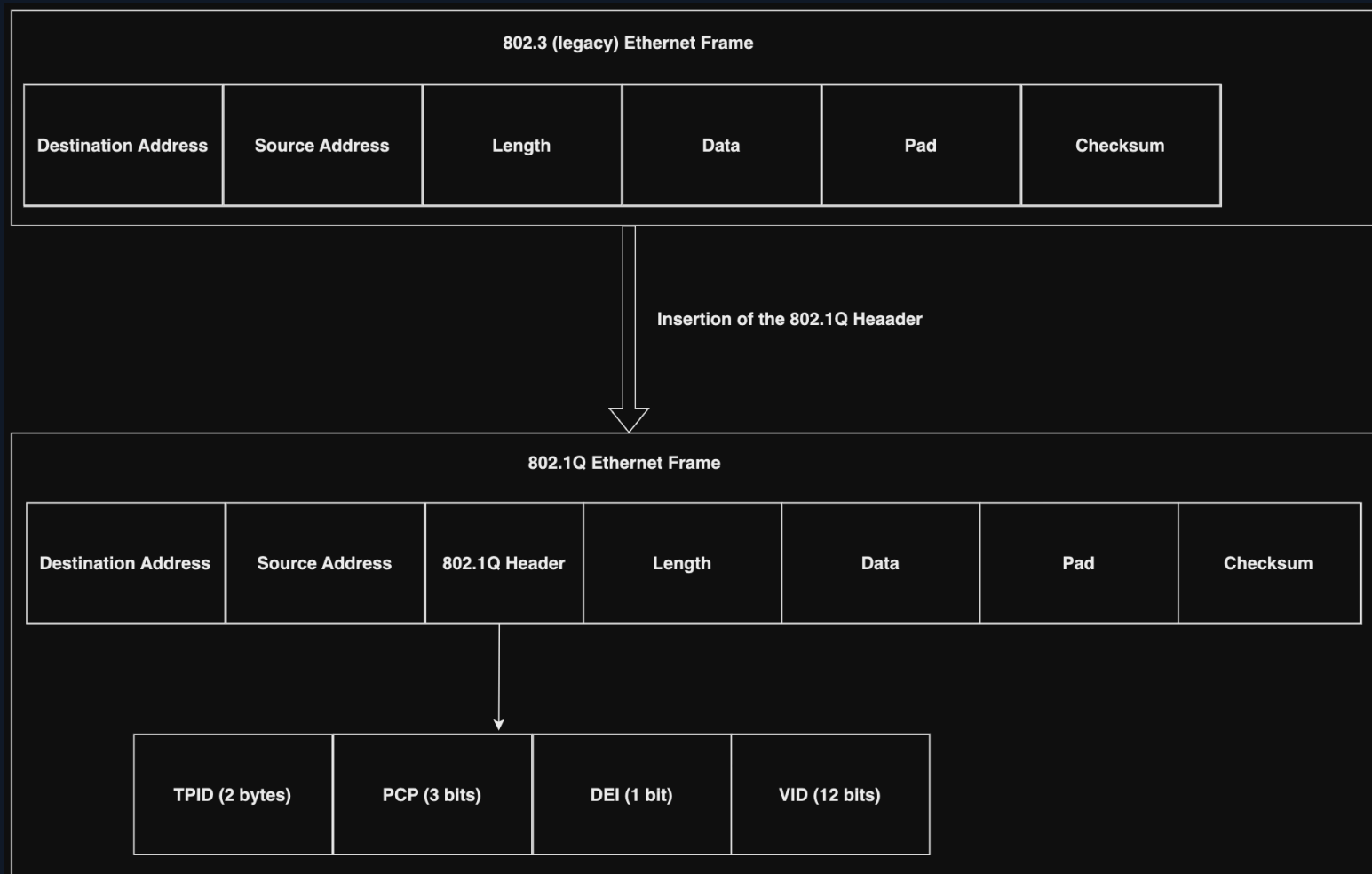
Standard **802.3 Ethernet** frames do not contain **VLAN** information; therefore, switches and other **VLAN**-enabled devices need a mechanism to keep track of all the **VLAN** information associated with a packet while traversing **VLAN**-enabled devices. Two main **trunking methods** are utilized to achieve this, **ISL** and **IEEE 802.1Q**.

Inter-Switch Link (ISL)

Inter-Switch Link (ISL) is a Cisco-proprietary protocol used for trunking between **VLAN**-enabled devices. Although **ISL** is one of the first **trunking methods** (predating **802.1Q**), it is deprecated and not as widely used in modern Cisco switches (and routers). Instead, most only support the widely adopted **802.1Q**. **ISL** encapsulated the entire **Ethernet** frame, including the original **Ethernet** header and the **VLAN** tag, adding its 26-byte header and 4-byte trailer.

IEEE 802.1Q

To ensure interoperability of VLAN technologies from the various network-equipments vendors, the Institute of Electrical and Electronics Engineers (IEEE) developed the 802.1Q specification in 1998. The IEEE 802 committee had to change the 802.3 Ethernet frame format by adding a pair of 2-byte fields, TPID and TCI (which consists of three subfields, PCP, DEI, and VID), resulting in a VLAN-compliant 802.1Q Ethernet frame.



Tag protocol identifier (TPID) is a 16-bit field always set to `0x8100` to identify the Ethernet frame as an `802.1Q`-tagged frame. Tag Control Information (TCI) is a 16-bit field containing Priority code point (PCP), Drop eligible indicator (DEI) (previously known as Canonical format indicator (CFI)), and VLAN identifier (VID). The main field concerning VLANs is VID , occupying the low-order 12-bits of TCI . Since it is 12 bits, there are $2^{12} = 4096$ possible values, but since `0` and `4095` are reserved, you can actually use `4094` VLAN IDs. Therefore, an `802.1Q`-tagged frame can contain information for `4094` VLANs ; the practice of inserting multiple `802.1Q` tags within a single packet is known as Double Tagging , introduced by `802.1ad`. VLAN tagging is the process of inserting VLAN information into an `802.1Q` Ethernet header , while VLAN untagging is the process of removing the VLAN information from an `802.1Q`-tagged Ethernet frame and forwarding the packet to the destined ports.

VLAN-Capable NICs

Some network interface cards (NICs) attached to computers/servers support VLAN tagging . Let us see how we can assign a VLAN ID to a NIC using Linux and Windows.

Assigning NICs a VLAN in Linux

In Linux, creating a VLAN is done by creating an interface on top of another, called a parent interface. This VLAN interface will tag packets with the assigned VLAN ID while returning packets will be untagged.

To assign a network adapter a VLAN in Linux, many tools can be used, such as `ip`, `nmcli`, and `vconfig` (deprecated). However, first, we need to ensure that the Kernel has the `802.1Q` module loaded:

shellsession

```
athane@htb[/htb]$ sudo modprobe 8021q
```

Subsequently, we can use `lsmod` to make sure `8021q` was loaded successfully:

shellsession

```
athane@htb[/htb]$ lsmod | grep 8021
```

8021q	40960	0
garp	16384	1 8021q
mrp	20480	1 8021q

Now, we need to find the name of the physical `Ethernet` interface that we will create the `VLAN` interface on top of, which is `eth0`:

shellsession

```
athane@htb[/htb]$ ip a
```

<SNIP>

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
```

```
    link/ether a6:ba:3b:08:3a:36 brd ff:ff:ff:ff:ff:ff
```

```
    altname enp0s3
```

```
    altname ens3
```

```
    inet 94.2X.5X.72/22 brd 94.237.51.255 scope global dynamic eth0
```

```
        valid_lft 83489sec preferred_lft 83489sec
```

```
    inet6 fe80::a4ba:3bff:fe08:3a36/64 scope link
```

```
        valid_lft forever preferred_lft forever
```

Then, we will use `vconfig` to create a new interface that is a member of the desired `VLAN`, `20`, for example, on top of `eth0`:

```
athane@htb[/htb]$ sudo vconfig add eth0 20
```

shellsession

Warning: vconfig is deprecated and might be removed in the future, please migrate to ip(route2) as soon as possible!

To use `ip` instead:

```
sudo ip link add link eth0 name eth0.20 type vlan id 20
```

shellsession

Either of these commands will make a new interface called `eth0.20@eth0`:

```
athane@htb[/htb]$ ip a
```

shellsession

<SNIP>

```
4: eth0.20@eth0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default  
qlen 1000  
    link/ether a6:ba:3b:08:3a:36 brd ff:ff:ff:ff:ff:ff
```

Then, based on the **subnet** assigned to the addresses with **VLAN 20** within the local network, we need to assign the interface an IP address and then start it:

```
athane@htb[/htb]$ sudo ip addr add 192.168.1.1/24 dev eth0.20
athane@htb[/htb]$ sudo ip link set up eth0.20
```

shellsession

At last, we can check whether the interface has changed states to up:

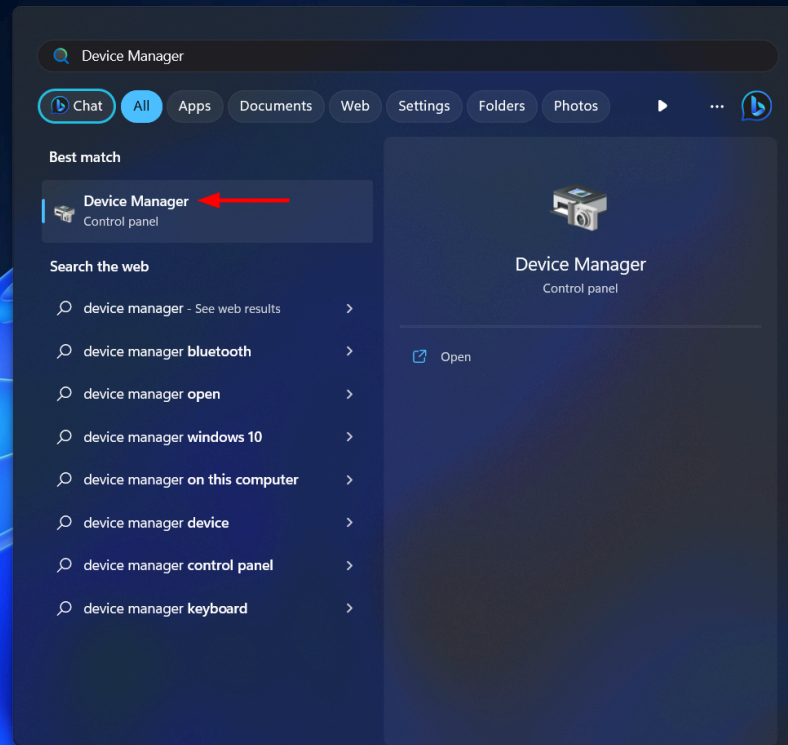
```
athane@htb[/htb]$ ip a | grep eth0.20

4: eth0.20@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    group default qlen 1000
    inet 192.168.1.1/24 scope global eth0.20
```

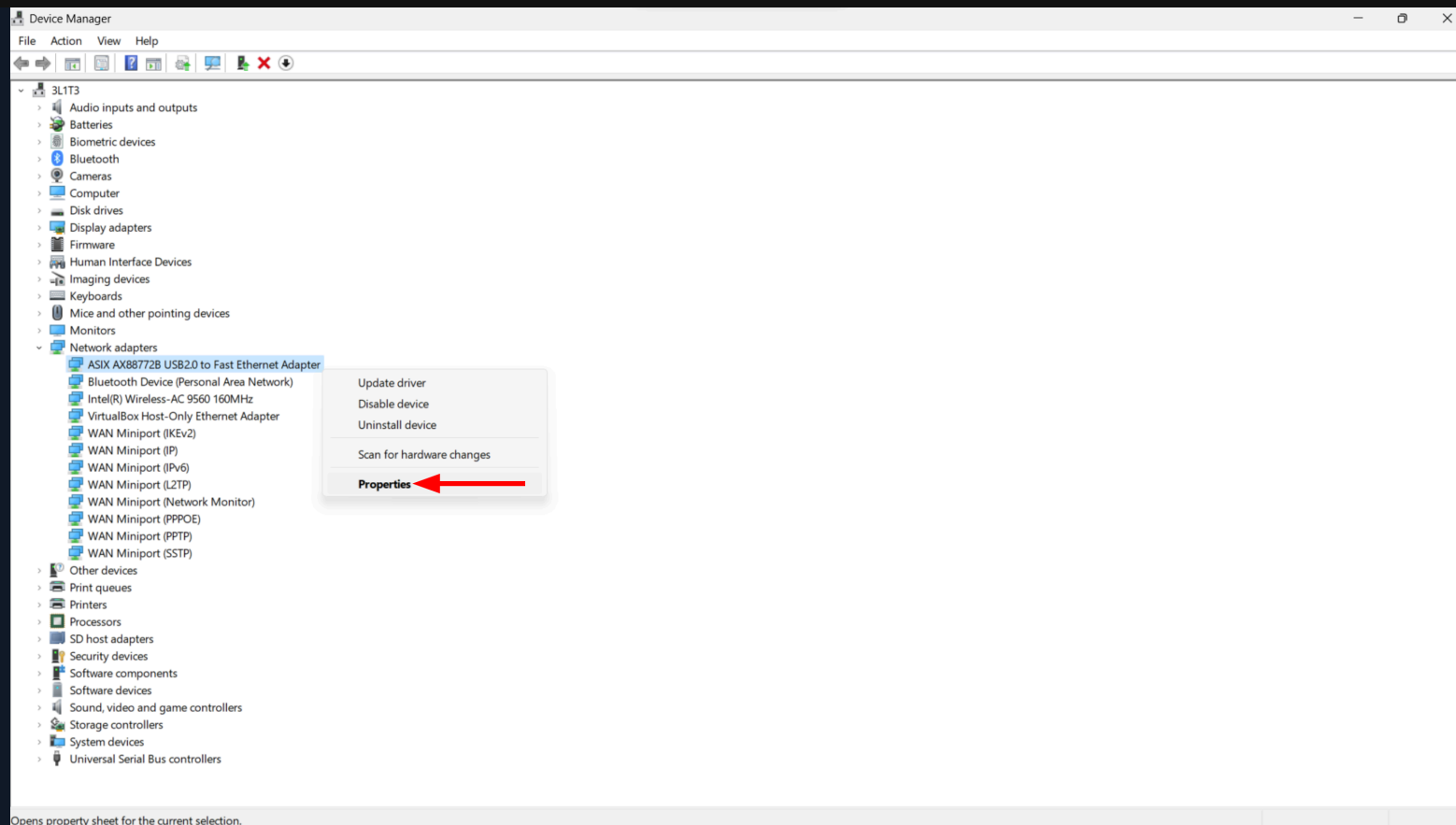
shellsession

Assigning NICs a VLAN in Windows

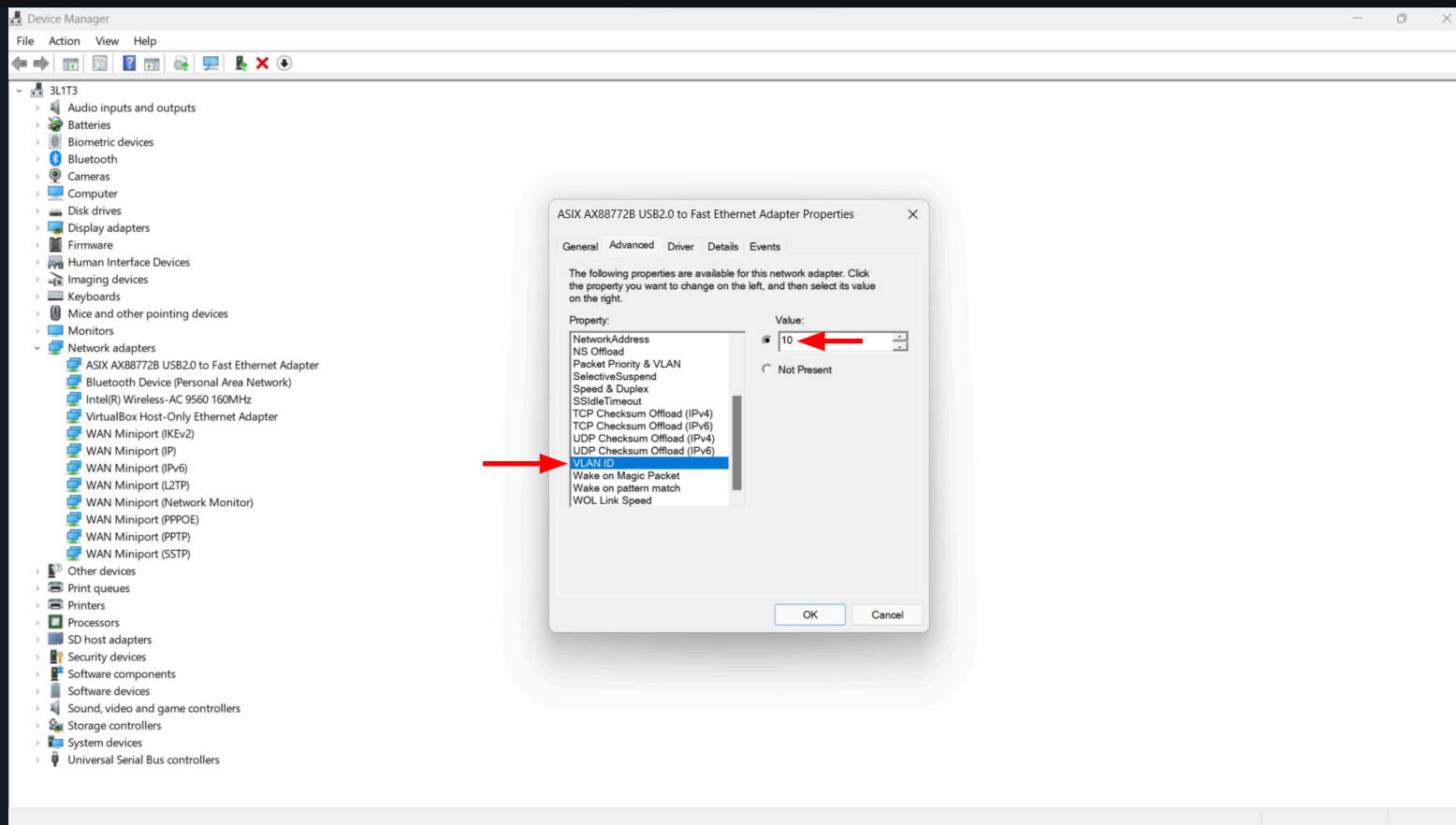
On Windows, to assign a **VLAN** for a physical network adapter that supports **VLAN tagging**, first we need to open **Device Manager**:



Then we need to click on **Properties** for the **Ethernet interface** we want to assign to a **VLAN** :



Within **Advanced** , there will be a **VLAN ID** property to which we can assign a value. After clicking **OK** , if the adapter supports assigning a **VLAN** , it will be set; otherwise, the window will close, and no **VLAN** tag will be added to any packets originating from this host:



Instead of relying on the GUI, we can use **PowerShell**. First, let us get the names of all the available physical network adapters using the **Get-NetAdapter** Cmdlet:

```

powershell-session
PS C:\> Get-NetAdapter | Format-Table -AutoSize

Name                                InterfaceDescription
----                                -
ifIndex Status                      MacAddress           LinkSpeed
-----
-----
VirtualBox Host-Only Network      VirtualBox Host-Only Ethernet Adapter

```

```

20 Up                                0A-00-27-10-42-15          1 Gbps
Ethernet 2                          ASIX AX88772B USB2.0 to Fast Ethernet
Adapter                             55 Up                      90-EB-78-14-21-7F      100
Mbps
Bluetooth Network Connection Bluetooth Device (Personal Area Network)
18 Disconnected 38-41-25-E8-DE-2D      3 Mbps
Wi-Fi                               Intel(R) Wireless-AC 9560 160MHz
12 Disconnected 8E-36-6A-7A-BA-6A 866.7 Mbps

```

Previously, we used `Device Manager` to assign `Ethernet 2` to `VLAN 10`; to retrieve the `VLAN` ID of the interface, we can use the `Get-NetAdapterAdvancedProperty` Cmdlet with the `-DisplayName` flag along with `vlan id`:

```
PS C:\> Get-NetAdapterAdvancedProperty -DisplayName "vlan id"
```

powershell-session

Name	DisplayName	DisplayValue
RegistryKeyword	RegistryValue	
-----	-----	-----
Ethernet 2	VLAN ID	10
VLAN_ID	{10}	

We can also set the `VLAN` ID of a physical network address using the `Set-NetAdapter` Cmdlet along with the `VlanID` flag; this powerful Cmdlet can also be used to customize other properties of interfaces such as `MAC addresses`:


```
PS C:\> Set-NetAdapter -Name "Ethernet 2" -VlanID 10
```

However, remember that this operation only succeeds if the network interface supports this functionality; otherwise, `PowerShell` will throw an error indicating that the interface does not support it.

Analyzing VLAN Tagged Traffic

We can identify and analyze `VLAN` tagged traffic on a network with `Wireshark` using the `vlan` filter. For example, when analyzing a network packet dump, we can inspect packets with `802.1Q` tagging using the filter `vlan` :

The screenshot displays the Wireshark network protocol analyzer interface. The top toolbar includes standard file and editing icons. The packet list pane on the left shows a series of captured packets, with packet 195 (DHCPv6 Reply) selected. The packet details pane on the right provides a hierarchical view of the selected packet's structure, including the Ethernet II header, the 802.1Q Virtual LAN header (VLAN ID 10), and the Internet Protocol Version 4 header. The packet bytes pane at the bottom shows the raw hexadecimal and ASCII data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1659	189080689.2...	fe80::d621:22ff:fe...	ff02::1:2	DHCPv6	163	Solicit XID: 0xc92c99 CID: 000100011cd0bf26d42122765b79
1660	189080689.2...	fe80::d621:22ff:fe...	ff02::1:2	DHCPv6	195	Reply XID: 0xc92c99 CID: 000100011cd0bf26d42122765b79
2163	247148574.7...	fe80::21a:6c9f:fea...	ff02::9	RIPng	130	Command Response, Version 1
2164	247148574.8...	Cisco_a1:5a:9a	PVST+	STP	68	RST. Root = 24576/121/00:0a:8a:a1:5a:80 Cost = 0 Port = 0x8042
2166	247148575.1...	fe80::21a:6c9f:fea...	ff02::66	HSRPv2	138	Hello (state Standby)
2167	247148575.3...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/80/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2168	247148575.3...	192.168.10.1	224.0.0.9	RIPv2	130	Response
2170	247148575.7...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/10/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2171	247148575.7...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/20/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2172	247148575.7...	Cisco_a1:5a:9a	PVST+	STP	68	RST. Root = 32768/1/00:0a:8a:a1:5a:80 Cost = 0 Port = 0x8042
2175	247148575.8...	Cisco_a1:5a:9a	PVST+	STP	68	RST. Root = 32768/3/00:0a:8a:a1:5a:80 Cost = 0 Port = 0x8042
2176	247148575.8...	Cisco_a1:5a:9a	Cisco_a1:5a:9a	LOOP	64	Reply
2177	247148576.0...	192.168.121.254	224.0.0.102	HSRPv2	118	Hello (state Active)
2178	247148576.3...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/70/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2179	247148576.7...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/30/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2180	247148576.7...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/40/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2181	247148576.7...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/50/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2182	247148576.7...	Cisco_ae:31:99	PVST+	STP	68	RST. Root = 24576/60/00:21:1b:ae:31:80 Cost = 0 Port = 0x8048
2183	247148576.8...	Cisco_a1:5a:9a	PVST+	STP	68	RST. Root = 24576/121/00:0a:8a:a1:5a:80 Cost = 0 Port = 0x8042
2184	247148577.1...	192.168.121.253	224.0.0.102	HSRPv2	118	Hello (state Standby)

Frame 2177: 118 bytes on wire (944 bits), 118 bytes captured (944 bits) on interface unk
 Ethernet II, Src: Cisco 9f:f0:79 (00:00:0c:9f:f0:79), Dst: IPv4mcast 66 (01:00:5e:00:00:00:00:00)
 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 121
 000. = Priority: Best Effort (default) (0)
 ...0 = DEI: Ineligible
 0000 0111 1001 = ID: 121
 Type: IPv4 (0x0800)
 Internet Protocol Version 4, Src: 192.168.121.254, Dst: 224.0.0.102
 User Datagram Protocol, Src Port: 1985, Dst Port: 1985
 Cisco Hot Standby Router Protocol

Moreover, we can search for packets with a specific **VLAN** ID; for example, to search for packets having **VLAN 10**, we can use the filter **vlan.id == 10**:

The screenshot shows a Wireshark packet capture of STP (Spanning Tree Protocol) messages. The packet list on the left shows several STP messages from source 192.168.10.1 to destination 224.0.0.9. Packet 2238 is selected, and the details pane on the right shows the following information:

- Ethernet II, Src: Cisco_79:3f:11 (00:1e:7a:79:3f:11), Dst: IPv6mcast_09 (33:33:00:00:00:09)
- 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 10
 - 000. = Priority: Best Effort (default) (0)
 - ...0 = DEI: Ineligible
 - 0000 1010 = ID: 10
 - Type: IPv6 (0x86dd)
- Internet Protocol Version 6, Src: fe80::21e:7aff:fe79:3f11, Dst: ff02::9
- User Datagram Protocol, Src Port: 521, Dst Port: 521
- RIPng

The bottom status bar indicates: 802.1Q Virtual LAN (vlan), 4 bytes. Packets: 38642 · Displayed: 221 (0.6%) · Profile: Default.

Additionally, to enumerate the used **VLAN** IDs from a packet dump, we can utilize **tshark**:

```
athane@htb[/htb]$ tshark -r "The Ultimate PCAP v20221220.pcapng" -T fields -e vlan.id | sort -n -u
```

1
2
3

7
10
20
30
40
50
60
70
80
90
121
125
224

Security Implications and VLAN Attacks

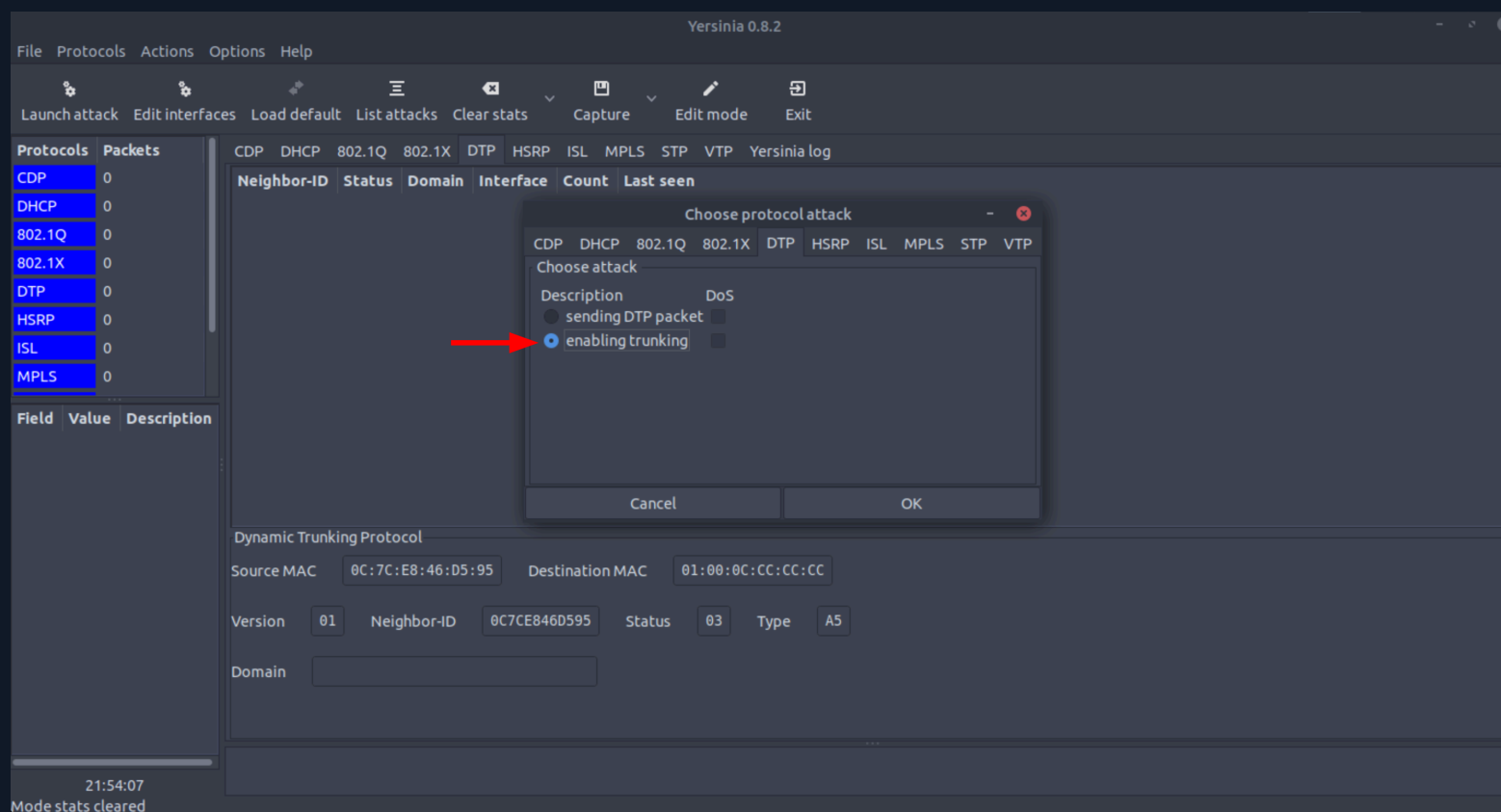
Regardless of improving a network's security posture, adversaries can still circumvent the defensive mechanisms put forth by **VLANs**. Although in modern switched networks, the utilization of **VLANs** brings numerous advantages (such as simplified network maintenance and improved performance), it also introduces potential security risks, leading to various **VLAN** attacks. It is essential to grasp the underlying methodologies of these attacks and implement practical mitigation approaches to safeguard networks.

VLAN Hopping

VLAN hopping attacks enable traffic from one **VLAN** to be seen by another **VLAN** without the aid of a router. It exploits Cisco's **Dynamic Trunking Protocol (DTP)**, a protocol used to

automatically negotiate the formation of a **trunk link** between two Cisco devices. An adversary needs to configure a host to mimic/act like a switch to take advantage of the automatic trunking port feature enabled by default on most switch ports. To exploit **VLAN hopping**, an adversary must be able to physically connect with a switch port that has **DTP** enabled. The adversary can abuse this connection by configuring a host connected to the switch on that specific port to spoof **802.1Q** signaling and the **DTP** packets. If successful, the switch will eventually establish a **trunk link** with the adversary's host, exposing the network packets, not only for a specific **VLAN**.

We can use tools such as **Yersinia** to perform **VLAN hopping** attacks:



Double-tagging VLAN Hopping

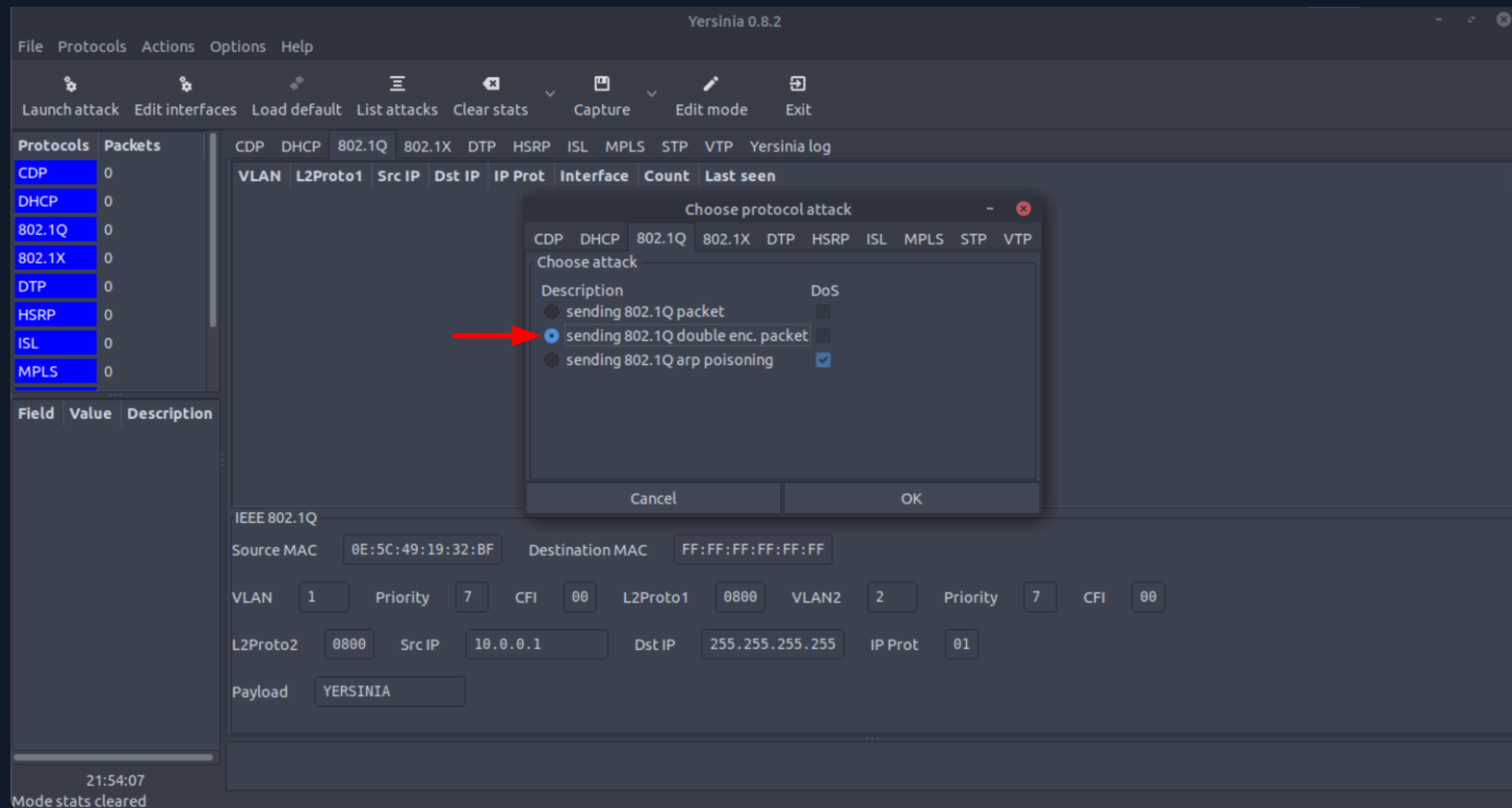
The **double-tagging VLAN hopping attack** is an increasingly more sophisticated attack against **VLANs**. Although **VLAN double-tagging** is a legitimate practice that entities such as **Internet Service Providers (ISPs)** utilize (they can use their **VLANs** internally while carrying traffic from clients that are already **VLAN tagged**), adversaries can also attempt to abuse it. In a **double-tagging VLAN hopping attack**, an adversary embeds a hidden **802.1Q** tag inside an **Ethernet** frame that already has an **802.1Q** tag, allowing the frame to go to a different **VLAN**, which the original **802.1Q** tag did not specify.

An adversary can carry out this attack following three steps. Bare in mind that this attack only works if the adversary is connected to a port residing in the same **VLAN** as the **native VLAN** of the trunk port:

1. The adversary sends a **double-tagged 802.1Q Ethernet** frame to the switch with the outer header having the **VLAN** ID of the adversary, which is the same as the native **VLAN** of the trunk port. Assume that the native **VLAN** is **VLAN 10** and that **VLAN 30** is the **VLAN** the adversary wants to reach, where the victim resides.
2. The outer 4-byte **802.1Q** tag arrives on the switch, and it is seen to be destined for **VLAN 10**, the native **VLAN**. After removing the **VLAN 10** tag, the frame is forwarded on all **VLAN 10** ports. On the trunk port, the **VLAN 10** tag is stripped (removed), and the packet is not re-tagged because it is part of the native **VLAN**. However, the **VLAN 30** tag is still intact (not stripped), and the first switch has not inspected it.
3. Subsequently, the switch will look only at the inner **802.1Q** tag that the adversary sent, and it decides that the frame must be forwarded for **VLAN 30**, which is the adversary's chosen

VLAN . Now, the second switch will either send the frame to the victim port directly or flood it, depending on whether there is an existing MAC address table entry for the victim host.

Scapy allows carrying out the **double-tagging VLAN hopping attack**, in addition to **Yersinia**:



VXLAN

We mentioned previously that the **VID** field within the '802.1Q' header inside an 'Ethernet' frame is only 12 bits, allowing for 4094 VLANs. While this number of VLANs might be sufficient for small

networks, more is needed for data centers and cloud service providers, which require extensive segmentation. Additionally, current Layer 2 networks utilize the **IEEE 802.1D Spanning Tree Protocol** (**STP**) to prevent network loops caused by redundant paths. However, some data center operators encounter limitations with **STP** , such as link blocking, which reduces available ports and prevents resiliency through multipathing. These challenges hinder network efficiency in virtualized environments that rely on Layer 2 physical infrastructure. A critical requirement in such environments is the seamless scalability of the Layer 2 network across the entire data center and even between data centers to allocate computing, networking, and storage resources efficiently. Nevertheless, traditional approaches like **STP** , while ensuring a loop-free topology, can deactivate many links, further exacerbating the problem.

RFC7348 offers a solution to these problems and limitations in Layer 2 networks by introducing **Virtual eXtensible Local Area Network** (**VXLAN**), which is essentially a 'Layer 2 overlay scheme on a Layer 3 network.' **VXLAN** is specifically designed to address the limitations of traditional Layer 2 networks and cater to the requirements of Layer 2 and Layer 3 data center network infrastructures in a multi-tenant environment with virtual machines (VMs). Operating over the existing networking infrastructure, **VXLAN** provides an innovative way to seamlessly extend a Layer 2 network. Its primary objective is to facilitate the scaling of Layer 2 networks across expansive data center landscapes, even spanning multiple physical data locations. Each **VXLAN** overlay is termed a **VXLAN segment** , ensuring that only VMs within the same VXLAN segment can communicate with each other, thus maintaining network isolation and security. A 24-bit segment ID, known as the **VXLAN Network Identifier** (**VNI**), uniquely identifies each VXLAN segment. Adopting VXLAN allows for the coexistence of 16 million **VXLAN** segments within the same administrative domain, providing scalability and flexibility for modern data centers and virtualized environments.

Cisco Discovery Protocol

Cisco Discovery Protocol (CDP) is a layer-2 network protocol from Cisco that is used by Cisco devices such as routers, switches, and bridges to gather information about other directly connected Cisco devices. This information can be used to discover and track the network's topology and help manage and troubleshoot the network. This protocol is usually enabled in Cisco devices, but it can be disabled if it is not needed or if it should be disabled for security reasons.

CDP Network Traffic

```
shellsession
22:14:11.563654 CDPv2, ttl: 180s, checksum: 0xebc1 (incorrect -> 0x8b71), length: 180
  Device-ID (0x01), length: 14 bytes: 'router.inlanefreight.loc'
  Addresses (0x02), length: 8 bytes:
    IPv4 (0x01), length: 4: 10.129.100.1
  Port-ID (0x03), length: 9 bytes: 'Ethernet0/0'
  Capability (0x04), length: 4: (0x00000010): Router
  Version String (0x05), length: 27 bytes: 'Cisco IOS Software, C880 Software'
  Platform (0x06), length: 26 bytes: 'Cisco 881 (MPC8300) processor'
```

The shown message contains information about the device itself, such as the device name, IP address, port name, and functionality of the router, as well as information about the operating system and hardware platform of the device. Besides, we can see in the first line from the **CDPv2** that we are dealing with the **Cisco Discovery Protocol**.

For comparison, we can look at another protocol called Spanning Tree Protocol (**STP**). The **STP** is a network protocol that ensures no loops in a network with multiple connections between

switches. There are no loops, and it prevents data packets from circulating in a loop and congesting the network.

STP Network Traffic

```
shellsession
22:14:11.563654 STP 802.1w, Rapid STP, Flags [Learn, Forward], bridge-id
8001.00:11:22:33:44:55.8000, length 43
    root-id 8001.AA:AA:AA:AA:AA:AA, cost 0, port-id 8001, message-age 0.00s, max-
age 20.00s, hello-time 2.00s, forward-delay 15.00s
```

In this example, we see that an **STP** message was sent containing information about the root switch, the MAC address of the root switch, the ID of the port over which the message was sent, and other configuration parameters such as the maximum aging time, hello time, and forward delay.



17 / 21 Sections



Mark Complete & Next